Vietnam National University Ho Chi Minh City
University Of Science
Falculty Of Information Technology

Lab 3 - MR Spark

Course: Introduction to Big Data

Students:
Van Thi Diem My (23127091)
Le Quoc Anh (23127150)
Tran Tuan Kiet (23127215)
Nguyen Phuong Thao (23127306)

Lecturers:
Dr. Nguyen Ngoc Thao
M.Sc. Huynh Lam Hai Dang
B.Sc. Tran Huy Ban

September 8, 2026

Contents

1 Team Information and Task Assignment **1**

2 Task 1-1 **1**
2.1 Problem Statement . . . 1
2.2 Problem Decomposition . . . 2
2.3 Implementation Strategy . . . 3
2.3.1 Key Design and Map-to-Buckets Assignment . . . 3
2.3.2 Combiner and Shuffle Optimization . . . 4
2.3.3 Complexity Analysis: Naive vs Optimized . . . 5
2.3.4 Tie-Breaking Logic . . . 6
2.4 Results and Validation . . . 7
2.4.1 Experimental Results and Validation . . . 7
2.4.2 Output Export and Reproducibility . . . 8
2.5 Benchmark . . . 10

3 Task 1-2 **11**
3.1 Problem Statement . . . 11
3.2 Problem Decomposition . . . 12
3.3 Implementation Strategy . . . 13
3.3.1 Preprocessing . . . 13
3.3.2 Job 1 - Global Style Qualification . . . 14
3.3.3 Job 2 - Variety Computation . . . 15
3.3.4 Job 3 - Median Computation . . . 16
3.3.5 Output Generation . . . 16
3.4 Results and Validation . . . 17
3.4.1 Execution Results . . . 17
3.4.2 Correctness Validation . . . 18
3.5 Benchmark . . . 19

4 Task 2-1 **21**
4.1 Problem Statement . . . 21
4.2 Problem Decomposition . . . 21
4.3 Implementation Strategy . . . 22
4.3.1 Job 1 - Global Promotion Qualification . . . 22
4.3.2 Job 2 - Per-Order Promotion and State Computation . . . 23
4.3.3 Job 3 - City Percentage Computation . . . 24

4.4 Results and Validation . . . 25
4.5 Benchmark . . . 26
4.6 Task-Specific Requirements . . . 28
4.6.1 Structured API Compliance . . . 28
4.6.2 Extended Execution Plan . . . 28
4.6.3 Exchange and Stage Analysis . . . 29
4.6.4 Single-File Parquet Output . . . 29

5 Task 2.2 **30**
5.1 Problem Statement . . . 30
5.1.1 Objective and scope . . . 30
5.1.2 Input data and intended output . . . 30
5.1.3 Formalising the query . . . 30
5.1.4 Assumptions and processing conventions . . . 31
5.2 Problem Decomposition . . . 32
5.2.1 Decomposition into elementary processing steps . . . 32
5.2.2 Rationale for this decomposition . . . 33
5.3 Implementation Strategy . . . 33
5.3.1 Environment and deployment architecture . . . 33
5.3.2 Project and source organisation . . . 34
5.3.3 Data preparation . . . 35
5.3.4 Implementing the approximate threshold . . . 36
5.3.5 Implementing the exact threshold . . . 36
5.3.6 Applying the threshold and computing shared statistics . . . 37
5.3.7 Implementing the evaluation and the export . . . 38
5.3.8 Build, run and reproduction . . . 38
5.4 Results and Validation . . . 39
5.4.1 Evidence that the application ran on YARN . . . 39
5.4.2 Group coverage and result consistency . . . 40
5.4.3 Schema and output file . . . 40
5.4.4 Validating the file with Spark local . . . 41
5.5 Percentile Method Comparison and Partition Strategy — Task 2-2 Specific Requirements . . . 42
5.5.1 Threshold differences and interpreting accuracy . . . 43
5.5.2 Analysis of groups with differing qualifying record sets . . . 44
5.5.3 Execution-time comparison . . . 44
5.5.4 Group size and partition strategy . . . 46
5.5.5 Conclusion . . . 47

References **48**

A Appendix **48**

1 Team Information and Task Assignment

No.	Full Name	Student ID	Assigned Tasks	Completion Rate
1	Van Thi Diem My	23127091	– Complete Section 2-1 – Contribute to report writing	100%
2	Le Quoc Anh	23127150	– Complete Section 2-2 – Contribute to report writing	100%
3	Tran Tuan Kiet	23127215	– Complete Section 1-1 – Contribute to report writing	100%
4	Nguyen Phuong Thao	23127306	– Complete Section 1-2 – Contribute to report writing	100%

Table 1: Team Task Assignment

2 Task 1-1

2.1 Problem Statement

The problem requires determining, for each state and each day d, the most frequently purchased size within a window of days preceding d. The window length depends on the total number of bought orders of the state across the entire dataset:

- If the total number of bought orders is greater than 10,000, then $L = 5$.
- Otherwise, $L = 10$.

The window for day d is $[d - L, d - 1]$, meaning that day d itself is not included.
A record is considered *bought* if its `Status` contains `"shipped"` and `Qty != 0`.
For each `(state, window_date)`, the answer is the size with the highest frequency. If multiple sizes share the same highest frequency, tie-breaking is applied sequentially based on the population variance of the purchased amount, followed by the lexicographical order of the size. The tie-breaking logic is discussed in detail in a later section.
The important assumptions and preprocessing rules are summarized in Table ??.

Attribute	Processing	Purpose
State	`UPPER(TRIM(state))`	Merge different representations of the same state into a consistent key, avoiding incorrect bought counts by state.
Date	Parse `MM-dd-yy`	Normalize dates into a consistent format to correctly identify and compute window dates.
Bought	`status.contains("shipped") && Qty != 0`	Identify valid records that are counted as bought.
Bought count	Each qualifying record is counted as 1	Used to determine whether the window length is 5 or 10 days.
Purchased amount	Use the `Amount` column	Used to compute variance during tie-breaking.
Amount null	Assume `0.0`	An implementation assumption used to handle missing values.
CSV	Parse as quoted CSV and verify exactly 24 columns	Avoid incorrect column splitting when a field contains commas inside quotation marks.

Table 2: Data preprocessing rules used in Task 1-1.

For the *purchased amount*, we interpret it as the `Amount` column because it directly represents the monetary amount of an order. The reference material explicitly identifies this as an ambiguity and recommends `Amount`. [1]
The treatment of missing `Amount` values as `0.0` is an **implementation assumption made by the group**, rather than a rule explicitly specified in the problem statement.

2.2 Problem Decomposition

The solution is decomposed into **three jobs**:
Job 0 — StateCountJob
Compute the total number of bought records for each state across the entire dataset in order to determine whether that state uses a 5-day or 10-day window.
The output of Job 0 contains only 46 rows, so it is provided to Job 1 through the **Distributed Cache**. Because the table is small, each mapper can load it into memory and directly determine whether a state uses a 5-day or 10-day window, avoiding an extra join or shuffle.

Job 1 — BucketAggregationJob
Map each bought record to all future window labels that the record belongs to, depending on whether $L = 5$ or $L = 10$, and then aggregate by `(state, window_date, size)`.

Job 2 — WinnerSelectionJob

Group all sizes belonging to the same `(state, date)` and apply the tie-breaking rules to select exactly one winner.

2.3 Implementation Strategy

2.3.1 Key Design and Map-to-Buckets Assignment

- **Job 1** uses the following key:

  `(state, window_date, size)`

  and value:

  `(count, sumAmount, sumAmountSquared)`

  A bought record occurring on day t affects the next L days. Therefore, the Mapper emits that record to all `window_date` values from $t+1$ to $t+L$. Each initial emission has the form:

  `(1, Amount, Amount`^{2}`)`

  For example, consider an order with:

  `State = GOA`
  `Date = 2022-04-05`
  `Size = M`
  $L = 10$

  The Mapper emits the following keys:

  ```
  (GOA, 2022-04-06, M)
  (GOA, 2022-04-07, M)
  ...
  (GOA, 2022-04-15, M)
  ```

  with the same corresponding value:

  `(1, Amount, Amount`^{2}`)`

- **Job 2** changes the key to:

  `(state, window_date)`

  and the value contains:

  `(size, frequency, sumAmount, sumAmount`^{2}`)`

  Instead of implementing a custom secondary sort, the group uses a separate reduction stage in Job 2. Job 1 first performs exact aggregation for each `(state, window_date, size)` bucket. Job 2 then changes the key to `(state, window_date)` so that the Reducer can receive all candidate sizes belonging to the same state and window date, and then perform winner selection based on frequency and the tie-breaking rules.

2.3.2 Combiner and Shuffle Optimization

Since the statistics `(count, sumAmount, sumAmountSquared)` only need to be accumulated, the same aggregation logic can be used for both the Combiner and the Reducer. The Combiner performs preliminary aggregation before the shuffle phase in order to reduce the amount of data transferred over the network to the Reducer, while the Reducer subsequently performs the final aggregation.

```
Map-Reduce Framework
        Map input records=128976
        Map output records=925395
        Map output bytes=38171245
        Map output materialized bytes=1275859
        Input split bytes=113
        Combine input records=925395
        Combine output records=27134
        Reduce input groups=27134
        Reduce shuffle bytes=1275859
        Reduce input records=27134
        Reduce output records=27134
```

Figure 1: Bucket emissions and shuffle reduction in Job 1.

Job 1 initially generated **925,395 bucket records**. The Combiner reduced these to **27,134 intermediate records** before the reduce phase.
The compression ratio is:

$$\frac{925395}{27134} \approx 34.1$$

which corresponds to approximately a **34-fold reduction**.

2.3.3 Complexity Analysis: Naive vs Optimized

Let:

- n be the number of bought records;
- w be the window length (5 or 10).

For example, suppose there is one order on `05/04` and its state uses a 10-day window. This order affects 10 future window dates, meaning that one input record produces 10 emitted records.
If there are n bought records and each record can be emitted at most w times, the total number of bucket emissions is:

$$n \times w$$

Therefore, the naive direct map-to-buckets approach has an emission complexity of:

$$O(nw)$$

On the actual dataset, this approach generated **925,395 intermediate records**.

The group's implementation still uses direct map-to-buckets, but adds a **Combiner** to reduce shuffle volume. The Combiner locally aggregates records having the same `(state, window_date, size)` before the data is transferred through the shuffle phase. Therefore, although the Mapper still generates **925,395 records** and the complexity of bucket assignment remains $O(nw)$, only **27,134 records** actually enter the Reduce phase. Thus, the Combiner does not reduce the number of Mapper emissions, but it significantly reduces the amount of data transferred between Map and Reduce.

Another theoretical optimization is the **difference array**. Instead of emitting one record for every day in the window, each bought record only needs to generate **two boundary events**: a `+1` event at the first affected day and a `-1` event at the day immediately after the affected interval ends. The values for individual days can then be reconstructed using prefix accumulation.
Since each record always generates only two emissions, the total number of emissions becomes:

$$2n = O(n)$$

With **109,566 bought records** in this dataset, the number of boundary emissions would be:

$$2 \times 109{,}566 = 219{,}132$$

Therefore, the difference-array approach can reduce the number of emissions from $O(nw)$ to $O(n)$.
However, the group does not implement this method and only analyzes it as a theoretical optimization because the difference-array approach requires an additional time-ordered prefix-accumulation step and makes the implementation more complex. Meanwhile, for this task where $w \leq 10$, direct mapping combined with a Combiner remains sufficiently simple, easy to validate, and already significantly reduces shuffle volume.

Approach	**Emissions per record when** $w = 10$	**This dataset**	**Observation**
Naive direct map-to-buckets	10	925,395	$O(nw)$
Difference array	2	219,132	$O(n)$ emissions
Direct + Combiner (implemented)	**still 10**	**925,395 → 27,134 after Combiner**	Mapper remains $O(nw)$, but shuffle is significantly reduced.

Table 3: Comparison of bucket-emission approaches.

2.3.4 Tie-Breaking Logic

Job 2 applies the following tie-breaking order:

higher frequency

↓ if tied

lower population variance

$\downarrow$ if still tied

lexicographically smaller size

Population variance, divided by N, is computed as:

$$\mathrm{Var}(X) = \frac{\sum x^2}{N} - \left(\frac{\sum x}{N}\right)^2$$

To compute population variance for tie-breaking, the group does not store the entire list of `Amount` values for each bucket in memory. Instead, each bucket maintains only three cumulative statistics: N, $\sum x$, and $\sum x^2$. These three values are sufficient to compute the population variance at the Reducer. This design reduces memory usage while allowing the data to be partially aggregated through the Mapper, Combiner, and Reducer. Therefore, this approach is more suitable for Big Data problems, where the number of records can be very large and storing all intermediate values in memory is inefficient.

If multiple sizes still have the same frequency and population variance, the group selects the lexicographically smallest size. In the implementation, `String.compareTo` is used for this final comparison step.

2.4 Results and Validation

2.4.1 Experimental Results and Validation

First, Job 0 was validated with the following results:

```
tuankiet@ubuntu1:~/Lab3/Task_1-1$ echo "===== JOB 0 VALIDATION ====="

echo "Number of states:"
hdfs dfs -cat /lab3/task1_1/state_counts/part-r-00000 | wc -l

echo "States with more than 10,000 bought orders:"
hdfs dfs -cat /lab3/task1_1/state_counts/part-r-00000 \
  | awk '$NF + 0 > 10000'

echo "Total usable bought records:"
hdfs dfs -cat /lab3/task1_1/state_counts/part-r-00000 \
  | awk '{sum += $NF} END {print sum}'
===== JOB 0 VALIDATION =====
Number of states:
2026-09-05 14:52:48,282 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
46
States with more than 10,000 bought orders:
2026-09-05 14:52:49,345 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
KARNATAKA       14950
MAHARASHTRA     19103
Total usable bought records:
2026-09-05 14:52:50,321 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
109566
tuankiet@ubuntu1:~/Lab3/Task_1-1$
```

Figure 2: Validation of Job 0.

After the filtering and normalization steps in the implementation, Job 0 recorded **109,566 usable bought records**, and this value was used by Job 1 as the basis for bucket assignment.

Job 0 produced 46 normalized states, and only **MAHARASHTRA (19,103)** and **KARNATAKA (14,950)** exceeded the threshold of 10,000 bought records. Therefore, these two states use a 5-day window, while the remaining states use a 10-day window. These results match the checkpoints provided in the reference slide. [1]
The final-output validation results are:

```
tuankiet@ubuntu1:~/Lab3/Task_1-1$ hdfs dfs -cat \
  /lab3/task1_1/final/part-r-00000 \
  | wc -l
2026-09-05 15:30:15,527 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
3697
tuankiet@ubuntu1:~/Lab3/Task_1-1$ hdfs dfs -cat \
  /lab3/task1_1/final/part-r-00000 \
  | tail -n +2 \
  | cut -d',' -f2 \
  | sort \
  | tail -n 1
2026-09-05 15:30:34,875 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2022-07-09
tuankiet@ubuntu1:~/Lab3/Task_1-1$ hdfs dfs -cat \
  /lab3/task1_1/final/part-r-00000 \
  | grep ',2022-03-31,'
2026-09-05 15:30:53,109 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
tuankiet@ubuntu1:~/Lab3/Task_1-1$ hdfs dfs -cat \
  /lab3/task1_1/final/part-r-00000 \
  | tail -n +2 \
  | awk -F',' '$3 == "M" {count++} END {print count}'
2026-09-05 15:31:03,372 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
1299
```

Figure 3: Final-output validation.

The final output contains **3,697 lines in total**, including **1 header row** and **3,696 data rows**, with the maximum `window_date` equal to **2022-07-09**. There are no rows for **2022-03-31**, and size `M` wins **1,299 windows**. These values match the checkpoints in the reference slide [1], indicating that the sliding-window boundaries and winner-selection logic were handled correctly.

2.4.2 Output Export and Reproducibility

Execution Guide The solution is fully implemented in Scala. Task 1-1 consists of three Scala MapReduce programs: `StateCountJob.scala`, `BucketAggregationJob.scala`, and `WinnerSelectionJob.scala`. Each source file was compiled and packaged into a separate runnable JAR, then executed sequentially as Hadoop MapReduce jobs on YARN.
The three jobs were executed as follows:

```
# Job 0
hadoop jar StateCountJob-fat.jar StateCountJob \
  /lab3/task1_1/input \
  /lab3/task1_1/state_counts

# Job 1
hadoop jar BucketAggregationJob-fat.jar BucketAggregationJob \
  /lab3/task1_1/input \
  /lab3/task1_1/state_counts/part-r-00000 \
  /lab3/task1_1/bucket_stats

```

```
# Job 2
hadoop jar WinnerSelectionJob-fat.jar WinnerSelectionJob \
  /lab3/task1_1/bucket_stats \
  /lab3/task1_1/final
```

```
Job completed successfully
```

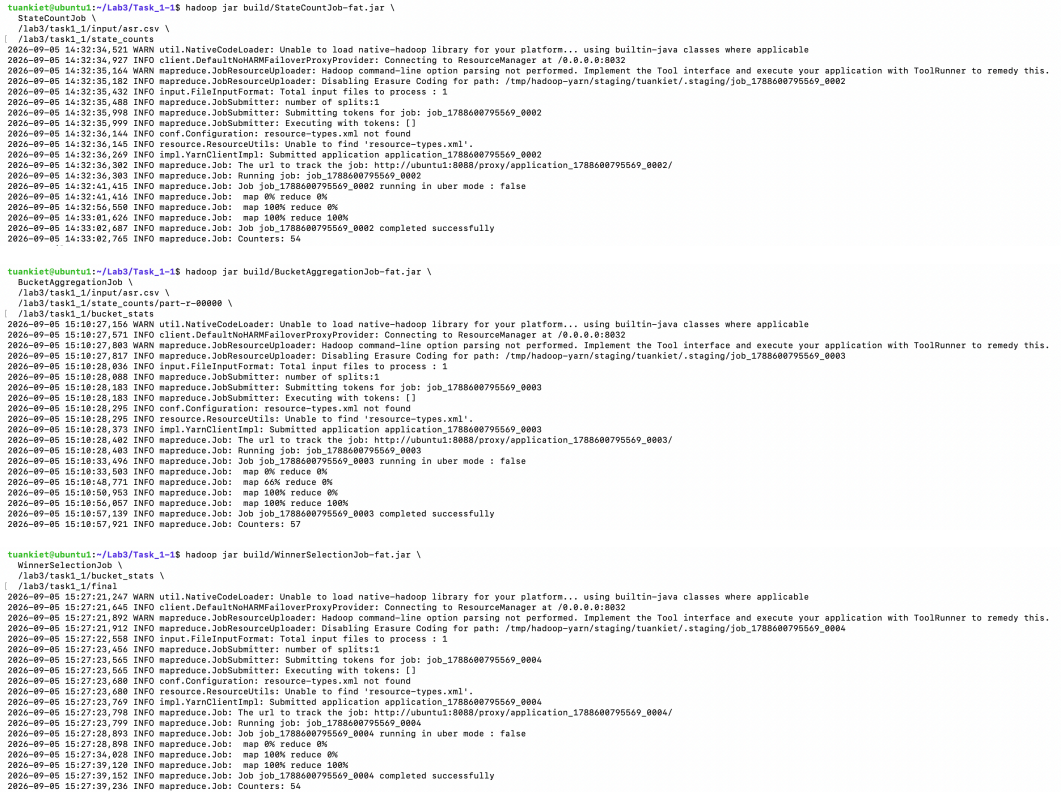

Figure 4: Successful execution of the three MapReduce jobs.

Output Export After Job 2 was completed, the final result was produced in HDFS as `part-r-00000`. The group then exported this file to the normal filesystem and renamed it to `Task_1-1.csv`, following the naming convention of the task while also making the result convenient to inspect and use outside the HDFS environment.

```
tuankiet@ubuntu1:~/Lab3/Task_1-1$ mkdir -p local_output
tuankiet@ubuntu1:~/Lab3/Task_1-1$ hdfs dfs -get \
  /lab3/task1_1/final/part-r-00000 \
  local_output/Task_1-1.csv
2026-09-05 15:31:41,108 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
tuankiet@ubuntu1:~/Lab3/Task_1-1$ ls -lh local_output/Task_1-1.csv
-rw-r--r-- 1 tuankiet tuankiet 153K Sep  5 15:31 local_output/Task_1-1.csv
tuankiet@ubuntu1:~/Lab3/Task_1-1$ head -n 10 local_output/Task_1-1.csv
state,window_date,most_bought_size,frequency,population_variance
ANDAMAN & NICOBAR,2022-04-02,M,1,0.0
ANDAMAN & NICOBAR,2022-04-03,S,2,5700.25
ANDAMAN & NICOBAR,2022-04-04,S,3,76126.8888888889
ANDAMAN & NICOBAR,2022-04-05,XS,3,10072.888888888876
ANDAMAN & NICOBAR,2022-04-06,XS,3,10072.888888888876
ANDAMAN & NICOBAR,2022-04-07,S,4,57160.5
ANDAMAN & NICOBAR,2022-04-08,XL,4,23102.75
ANDAMAN & NICOBAR,2022-04-09,XL,5,18957.440000000002
ANDAMAN & NICOBAR,2022-04-10,XL,5,18957.440000000002
tuankiet@ubuntu1:~/Lab3/Task_1-1$ wc -l local_output/Task_1-1.csv
3697 local_output/Task_1-1.csv
```

Figure 5: Exported `Task_1-1.csv` on the normal filesystem.

The exported `Task_1-1.csv` file was validated again on the normal filesystem. The file has the expected schema:

```
state,window_date,most_bought_size,frequency,population_variance
```

It contains **3,697 lines including the header**, corresponding to **3,696 data rows**. This result matches the values previously validated from the HDFS output.

2.5 Benchmark

To evaluate the execution performance and resource consumption of Task 1-1, each MapReduce job was executed five times under the same Hadoop/YARN environment and input dataset. For each run, three metrics were recorded: elapsed execution time, memory-seconds, and vcore-seconds. The mean and standard deviation were then calculated across the five runs.

Run	Time elapsed (s)	Memory-seconds	vcore-seconds
1	26.35	60,510	36
2	25.69	60,961	35
3	26.62	61,502	37
4	25.66	58,579	34
5	26.35	62,539	37
Mean	**26.13**	**60,818.20**	**35.80**
Std Dev	**0.43**	**1,462.71**	**1.30**

Table 4: Benchmark results for Job 0 of Task 1-1.

Run	Time elapsed (s)	Memory-seconds	vcore-seconds
1	28.29	65,676	39
2	30.50	68,750	41
3	30.34	70,420	42
4	29.22	66,897	39
5	28.89	66,719	40
Mean	**29.45**	**67,692.40**	**40.20**
Std Dev	**0.95**	**1,884.43**	**1.30**

Table 5: Benchmark results for Job 1 of Task 1-1.

Run	Time elapsed (s)	Memory-seconds	vcore-seconds
1	18.44	31,998	17
2	16.32	29,515	16
3	16.20	28,450	15
4	18.72	32,736	17
5	18.73	33,449	19
Mean	**17.68**	**31,229.60**	**16.80**
Std Dev	**1.30**	**2,147.76**	**1.48**

Table 6: Benchmark results for Job 2 of Task 1-1.

Overall, Job 1 has the highest average execution time and resource consumption among the three jobs, with an average runtime of **29.45 seconds**, **67,692.40 memory-seconds**, and **40.20 vcore-seconds**. Job 0 follows with an average runtime of **26.13 seconds**, while Job 2 is the least resource-intensive, with an average runtime of **17.68 seconds**. The relatively small standard deviations in execution time indicate that the benchmark results are generally stable across the five runs.

3 Task 1-2

3.1 Problem Statement

Task 1-2 requires computing the **state-level median variety for each month**. The variety of a style is defined as the number of distinct SKUs associated with that style within a specific time interval and geographical region.
For month m, state s, and style t, we define

$$V(m, s, t) = |\{sku \mid sku \text{ belongs to style } t \text{ in month } m \text{ and state } s\}| .$$

The required output for each $(month, state)$ pair is the median of the variety values of all eligible styles:

$$M(m, s) = \text{median}\left(\{V(m, s, t) \mid t \text{ is eligible}\}\right) .$$

An important ambiguity is the phrase "style which has served a size of at least XXL". It may be interpreted either within each $(month, state)$ group or globally over the entire dataset.
In this implementation, we adopt the **global interpretation**: a style is considered eligible if it contains at least one record with size $\geq$ `XXL` anywhere in the complete dataset. Once qualified, all occurrences of that style may contribute to its variety in every $(month, state)$ group where the style appears.
Formally, the set of globally eligible styles is

$$G = \{t \mid \exists r : r.style = t \wedge r.size \geq XXL\} .$$

For example, if a style serves `XXL` in Karnataka in May but only smaller sizes in Maharashtra in April, it is still included when calculating the April-Maharashtra result.
The size condition must also be evaluated semantically rather than lexicographically. The implementation therefore assigns an explicit rank:

$$XS = 1,\ S = 2,\ M = 3,\ L = 4,\ XL = 5,\ XXL = 6,\ 3XL = 7,\ 4XL = 8, \ldots$$

and considers a size eligible when $sizeRank(size) \geq 6$. This prevents values such as `3XL` from being incorrectly ordered with respect to `XXL`.

3.2 Problem Decomposition

The task is decomposed into the following main processing steps:

1. **Read and preprocess the input data:** read the Amazon Sale Report, skip the header, and extract the fields required for this task, including `Date`, `ship-state`, `Style`, `SKU`, and `Size`.

2. **Normalize the required attributes:** parse dates from `MM-dd-yy` into monthly keys in the `yyyy-MM` format, normalize state names, styles, and SKUs, and map size values to their semantic ranks.

3. **Identify globally eligible styles:** scan the complete dataset and determine the set of styles that have served size `XXL` or larger at least once anywhere in the dataset.

4. **Filter records by global style eligibility:** retain records whose styles belong to the globally eligible style set.

5. **Group records by month, state, and style:** organize the remaining records using $(month, state, style)$ as the grouping key so that all SKUs belonging to the same style in the same month and state are processed together.

6. **Compute style variety:** for each $(month, state, style)$ group, count the number of distinct SKUs: $V(m, s, t) = |distinct(SKU)|$.

7. **Regroup variety values by month and state:** after the style-level variety has been computed, remove the style component from the key and group the resulting variety values by $(month, state)$.

8. **Compute the state-level median variety:** sort the style-level variety values within each $(month, state)$ group and compute their median. For an even number of values, the median is the average of the two middle values.

9. **Export the final result:** convert the final MapReduce output into a single CSV file with schema `month,state,median_variety` and write it to the local filesystem.

These processing steps are implemented using three MapReduce jobs:

$$\underbrace{\text{Global Style Qualification}}_{\text{Job 1}} \rightarrow \underbrace{\text{Variety Computation}}_{\text{Job 2}} \rightarrow \underbrace{\text{Median Computation}}_{\text{Job 3}}.$$

3.3 Implementation Strategy

3.3.1 Preprocessing

Only five fields are required for this task:

Field	Usage
`Date`	Month extraction
`ship-state`	State grouping
`Style`	Style qualification and grouping
`SKU`	Variety computation
`Size`	XXL eligibility check

The header row is skipped before processing. A quote-aware CSV parser is used instead of a simple comma-based split because some fields, such as `promotion-ids`, may contain commas inside quotation marks. Each valid record is expected to contain exactly 24 columns.

Dates follow the `MM-dd-yy` format and are converted to monthly keys in the `yyyy-MM` format. For example: `04-30-22` → `2022-04`.
State values are trimmed, repeated whitespace is collapsed, and the result is converted to uppercase. Style and SKU identifiers are also trimmed and uppercased to ensure that equivalent values are grouped under the same key.
The condition "size at least XXL" is evaluated using semantic size ordering rather than lexicographical comparison. The implementation assigns explicit ranks to the size values, for example:

```
case "XL"  => Some(5)
case "XXL" => Some(6)
case s if s.matches("[2-9][0-9]*XL") =>
  Some(s.stripSuffix("XL").toInt + 4)
```

A size is considered eligible when its rank is at least that of `XXL`. This ensures that values such as `3XL` and `4XL` are correctly treated as larger than `XXL`.
Malformed CSV rows, invalid dates, and records with missing fields required for grouping are skipped and tracked using Hadoop counters.

3.3.2 Job 1 - Global Style Qualification

The first job determines the globally eligible style set G. The mapper extracts the style and size and emits the style only when its size is at least `XXL`:

```
val style = normalizeStyle(cols(STYLE_IDX))
val size  = cols(SIZE_IDX)

if (style.nonEmpty && isAtLeastXXL(size)) {
  outKey.set(style)
  context.write(outKey, NullWritable.get())
}
```

The logical mapper output is therefore $(style, Null)$. The value is unnecessary because this stage only needs to determine whether a style belongs to the eligible set.
The reducer removes duplicate styles:

```
context.write(key, NullWritable.get())
```

The same reducer is also used as a combiner so that repeated occurrences of the same qualifying style can be removed before the shuffle phase, reducing unnecessary intermediate

data transfer.

3.3.3 Job 2 - Variety Computation

Job 1 outputs only the distinct set of globally eligible styles. This list is distributed to Job 2 mappers through Hadoop's distributed cache and loaded into a `HashSet`, allowing efficient membership checks.
A mapper tests eligibility using

```
if (!globallyValidStyles.contains(style)) {
  return
}
```

with expected $O(1)$ lookup time.
For each eligible record, the mapper constructs the key $(month, state, style)$ and emits the associated SKU:

```
outKey.set(s"$month|$state|$style")
outValue.set(sku)
context.write(outKey, outValue)
```

Thus, the logical output is

$$((month, state, style), SKU).$$

This composite key is required because variety must be computed independently for each style within each month and state. Hadoop's shuffle groups all SKU values with the same composite key at the same reducer.
A combiner locally removes repeated SKUs before the shuffle phase because duplicate SKU occurrences do not affect the distinct count. This reduces the amount of intermediate data transferred to the reducers.
The reducer then computes $V(m, s, t) = |distinct(SKU)|$ using a `HashSet`.
After variety is obtained, the style is removed from the output key:

```
val variety = uniqueSkus.size

outKey.set(s"$month|$state")
outValue.set(variety)
context.write(outKey, outValue)
```

Therefore, Job 2 transforms

$$(month, state, style)$$

into

$$(month, state) \rightarrow variety.$$

This prepares the records for the final median calculation.

3.3.4 Job 3 - Median Computation

Job 3 receives one variety value for each globally eligible style appearing in a month-state group. For each $(month, state)$ key, the reducer sorts all variety values:

```
val sorted = values.asScala.map(_.get()).toArray.sorted
val n = sorted.length
```

The median is then computed as

$$median = \begin{cases} x_{\lfloor n/2 \rfloor}, & n \text{ is odd}, \\ \dfrac{x_{n/2-1} + x_{n/2}}{2}, & n \text{ is even}. \end{cases}$$

The corresponding implementation is:

```
val median =
  if (n % 2 == 1) {
    sorted(n / 2).toDouble
  } else {
    (sorted(n / 2 - 1).toDouble +
     sorted(n / 2).toDouble) / 2.0
  }
```

A `DoubleWritable` is used because even-sized groups may produce fractional median values.

3.3.5 Output Generation

The original input and all intermediate MapReduce results are stored in HDFS. After Job 3 finishes, the `writeSingleCsv` function converts Hadoop's reducer output into one normal CSV file with the schema

```
month,state,median_variety
```

The application is executed with a local final output path such as

`file:///home/thao2005/Lab3/Task_1-2.csv`

where the `file:///` scheme explicitly selects the local filesystem.
Therefore, the storage flow is

HDFS input → HDFS intermediate outputs → local Task_1-2.csv.

3.4 Results and Validation

3.4.1 Execution Results

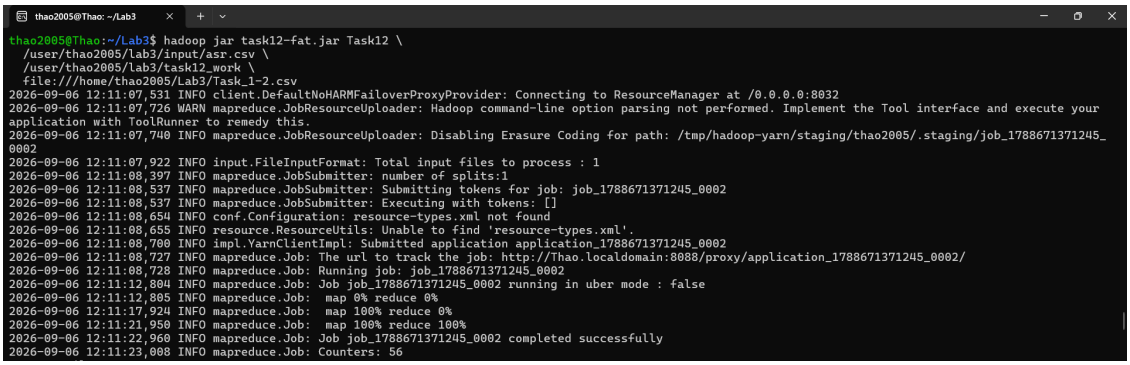

Figure 6: Successful execution of Job 1 - Global Style Qualification.

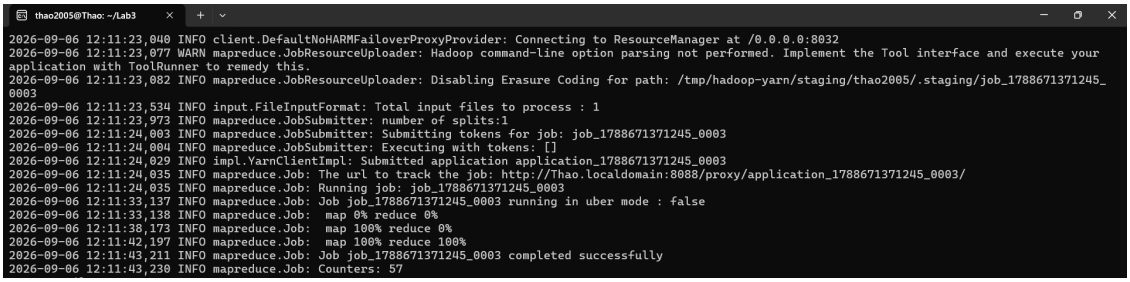

Figure 7: Successful execution of Job 2 - Variety Computation.

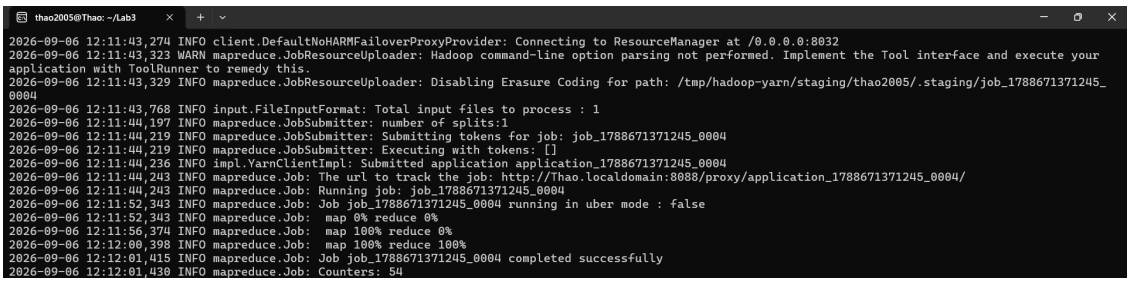

Figure 8: Successful execution of Job 3 - Median Computation.

The main execution results are summarized in Table 7.

Table 7: Task 1-2 execution and validation results

Quantity	**Result**
Valid data records (excluding header)	128,975
Records with size $\geq$ XXL	34,627
Globally eligible styles	1,103
Records processed by Job 2	127,747
$(month, state, style)$ groups	31,395
Final $(month, state)$ groups	145
Duplicate final keys	0
Missing output values	0

3.4.2 Correctness Validation

1. Output structure validation. The final output was first checked to ensure that every record followed the required schema:

```
month,state,median_variety
```

The final result contained 145 distinct $(month, state)$ keys. No duplicate keys and no missing output values were found:

$$\text{duplicate keys} = 0, \qquad \text{missing values} = 0.$$

This confirms that each month-state group is represented exactly once in the final output.

2. Intermediate result consistency. The number of records produced at each processing stage was also checked against the expected transformation of the three MapReduce jobs. Starting from 128,975 valid data records, Job 1 identified 34,627 records whose size is at least `XXL`. After duplicate styles were removed, 1,103 globally eligible styles remained.
Job 2 then processed records belonging to these globally eligible styles and produced 31,395 distinct $(month, state, style)$ groups. Each group corresponds to one style-level variety value

$$V(m, s, t) = |distinct(SKU)| .$$

Job 3 grouped these 31,395 variety values by $(month, state)$ and reduced them to 145 final month-state groups.
Therefore, the observed processing flow was

$$128{,}975 \rightarrow 1{,}103 \text{ eligible styles} \rightarrow 31{,}395\ (month, state, style) \text{ groups} \rightarrow 145\ (month, state) \text{ groups}.$$

These counts are consistent with the intended decomposition of the task.

3. Verification of the median calculation. The validation also included groups with both odd and even numbers of style-level variety values. For an odd-sized group, the middle value after sorting is selected directly. For an even-sized group, the median is computed as the average of the two middle values:

$$median = \begin{cases} x_{\lfloor n/2 \rfloor}, & n \text{ is odd}, \\ \dfrac{x_{n/2-1} + x_{n/2}}{2}, & n \text{ is even}. \end{cases}$$

For example, the group (2022-06, WEST BENGAL) produced a median variety of $\boxed{1.5}$.
The fractional result confirms that the implementation correctly handles an even number of style-level variety values by averaging the two middle values, rather than applying integer truncation.
Overall, the structural checks, intermediate counts, and edge-case verification consistently confirm the correctness of the Task 1-2 implementation.

3.5 Benchmark

Table 8: Benchmark results for Job 1 of Task 1-2.

Run	Time elapsed (s)	Memory-seconds	vcore-seconds
1	15.017	50,797	27
2	14.052	48,484	25
3	15.819	52,123	27
4	15.163	51,103	26
5	15.503	52,100	27
Mean	**15.111**	**50,921.40**	**26.40**
Std Dev	**0.669**	**1,485.12**	**0.894**

Table 9: Benchmark results for Job 2 of Task 1-2.

Run	Time elapsed (s)	Memory-seconds	vcore-seconds
1	18.558	50,307	27
2	19.147	52,732	28
3	25.328	65,229	35
4	20.755	56,399	30
5	15.986	52,449	28
Mean	**19.955**	**55,423.20**	**29.60**
Std Dev	**3.459**	**5,902.75**	**3.209**

Table 10: Benchmark results for Job 3 of Task 1-2.

Run	Time elapsed (s)	Memory-seconds	vcore-seconds
1	16.927	48,766	26
2	16.059	45,026	24
3	16.668	54,233	29
4	12.926	46,169	25
5	16.812	48,191	26
Mean	**15.878**	**48,477.00**	**26.00**
Std Dev	**1.684**	**3,554.33**	**1.871**

Table 11: End-to-end execution time of Task 1-2.

Run	Total execution time (s)
1	63.053
2	60.475
3	75.914
4	64.189
5	64.800
Mean	**65.686**
Std Dev	**5.952**

The end-to-end execution time includes the execution of all three MapReduce jobs as well as the overhead between jobs, such as intermediate HDFS operations and final CSV generation.

Table 12: MapReduce processing counters for Task 1-2.

Job	Map Input Records	Map Output Records	Reduce Output Records	Shuffle Bytes	Spilled Records
Job 1	128,976	34,627	1,103	10,121	2,206
Job 2	128,976	127,747	31,395	2,865,692	136,324
Job 3	31,395	31,395	145	761,279	62,790

The MapReduce counters provide additional insight into the performance differences among the three jobs. Job 1 significantly reduces the input from 128,976 records to 34,627 mapper outputs and generates only 10,121 bytes of shuffle data. In contrast, Job 2 retains most of the input records and produces approximately 2.87 MB of shuffle data together with 136,324 spilled records. This larger volume of intermediate data is consistent with Job 2 having the highest average execution time and resource consumption.

Job 3 processes the 31,395 style-level variety records produced by Job 2 and reduces them to 145 final month-state results. Its intermediate data volume is lower than that of Job 2, although it still requires a shuffle to group the variety values by month and state for median computation.

4 Task 2-1

4.1 Problem Statement

The objective of Task 2-1 is to calculate, for each city, the percentage of Cancelled orders using the Standard service level that satisfy both of the following conditions:

1. The order possesses at least three temporally valid promotions. All promotions are counted, including promotions issued by Amazon.

2. The order amount is lower than the average amount of Merchant-fulfilled orders with `Courier Status = Shipped` in the associated state.

A promotion is temporally valid when the number of days between its first and last appearances in the complete dataset is at least two:

$$\text{datediff}(\text{lastAppearance}, \text{firstAppearance}) \geq 2. \tag{1}$$

Let N_c be the number of Cancelled and Standard rows in city c, and let Q_c be the number of those rows satisfying both business conditions. The required percentage is

$$\text{percentage}(c) = 100 \times \frac{Q_c}{N_c}. \tag{2}$$

The denominator is therefore *not* every Standard order. It contains only Cancelled and Standard rows. Rows without promotions must remain in the denominator and receive a valid-promotion count of zero.
The input file is stored in HDFS at:

`/user/vandiemmy/lab3/input/Amazon Sale Report.csv`

The implementation must use Spark DataFrame/Dataset operations without direct SQL query strings and must write one submission file named `Task_2-1.parquet`.

4.2 Problem Decomposition

The processing pipeline is decomposed into the following operations:

1. Read the 24-column CSV using a static schema. Parse `Date` with the `MM-dd-yy` format and cast `Amount` to `double`.

2. Normalize comparison columns. Text filters use `lower(trim(...))`; city and state use `upper(trim(...))`.

3. Split `promotion-ids` on commas, remove empty values, retain Amazon-issued promotions, and explode the array into one row per promotion occurrence.
4. Group by promotion identifier and calculate its first and last appearance dates. Keep identifiers whose lifetime is at least two days.
5. Join the valid identifiers back to the exploded occurrences, group by `Order ID`, and retain orders with at least three valid promotion occurrences.
6. Filter Merchant-fulfilled rows whose courier status is exactly Shipped, then compute the average non-null amount by normalized state.
7. Build the denominator by filtering to Cancelled and Standard rows. Remove rows with a null city because a null value is not a valid output city.
8. Left-join promotion counts and state averages to the denominator. Mark a row as qualifying only when both business conditions hold.
9. Group by city and calculate the percentage. Select only `city` and `percentage` for the final output.

The left joins are essential. An inner join with the promotion-count table would remove orders without promotions and incorrectly shrink the denominator.

4.3 Implementation Strategy

The following three logical jobs describe the implementation. They should not be confused with Spark runtime job IDs, because Adaptive Query Execution (AQE) may create additional runtime jobs.

4.3.1 Job 1 - Global Promotion Qualification

The first job derives a global lifetime for every promotion. Promotion strings are split and exploded, after which `min` and `max` are computed over the complete dataset.

```
val promotionAppearances = orders
  .select(
    col("order_id"),
    col("order_date"),
    explode(col("promotion_ids")).as("promotion_id")
  )

val validPromotions = promotionAppearances
```

```
  .groupBy("promotion_id")
  .agg(
    min("order_date").as("first_appearance"),
    max("order_date").as("last_appearance")
  )
  .filter(
    datediff(col("last_appearance"),
             col("first_appearance")) >= 2
  )
  .select("promotion_id")
```

This operation finds 284 promotion identifiers, of which 185 are temporally valid. The validity calculation is global rather than city-specific or state-specific.

4.3.2 Job 2 - Per-Order Promotion and State Computation

The valid-promotion table is joined to the exploded occurrences. We count valid occurrences per `Order ID`; 30,503 order IDs have a count of at least three.

```
val validPromotionCounts = promotionAppearances
  .join(validPromotions, Seq("promotion_id"), "inner")
  .groupBy("order_id")
  .agg(count("promotion_id").as("valid_promotion_count"))
  .filter(col("valid_promotion_count") >= 3)
```

In parallel, the state benchmark is computed from Merchant-fulfilled rows with an exact courier status of Shipped:

```
val stateMerchantShippedAverages = orders
  .filter(
    col("fulfilment") === "merchant" &&
    col("courier_status") === "shipped" &&
    col("state").isNotNull &&
    col("amount").isNotNull
  )
  .groupBy("state")
  .agg(
    avg("amount").as("state_merchant_shipped_average")
  )
```

Normalizing state names with `upper(trim(...))` produces 40 non-null state thresholds. Without normalization, casing variants such as `Delhi`, `DELHI`, and `delhi` would be treated as different states.

4.3.3 Job 3 - City Percentage Computation

The denominator is created before the joins. The following filter is the first part of `evaluatedStandardOrders` in the submitted source:

```
orders
  .filter(
    col("service_level") === "standard" &&
    col("status").contains("cancelled") &&
    col("city").isNotNull
  )
```

Promotion counts and state averages are attached using left joins. A null promotion count is converted to zero. For explicit null handling, a null order amount is converted to zero only for the comparison.

```
val evaluatedStandardOrders = orders
  .filter(
    col("service_level") === "standard" &&
    col("status").contains("cancelled") &&
    col("city").isNotNull
  )
  .join(validPromotionCounts, Seq("order_id"), "left")
  .withColumn(
    "valid_promotion_count",
    coalesce(col("valid_promotion_count"), lit(0L))
  )
  .join(stateMerchantShippedAverages, Seq("state"), "left")
  .withColumn(
    "is_qualifying",
    col("valid_promotion_count") >= 3 &&
    col("state_merchant_shipped_average").isNotNull &&
    coalesce(col("amount"), lit(0.0)) <
      col("state_merchant_shipped_average")
  )

val result = evaluatedStandardOrders
  .groupBy("city")
  .agg(
    count(lit(1)).as("cancelled_standard_orders"),
    sum(when(col("is_qualifying"), lit(1L))
      .otherwise(lit(0L))).as("qualifying_orders")
  )
  .withColumn(
```

```
    "percentage",
    col("qualifying_orders").cast(DoubleType) /
      col("cancelled_standard_orders").cast(DoubleType) * 100.0
  )
  .select("city", "percentage")
```

The result is reduced to one partition and written as Parquet. The single `part-*.snappy.parquet` file is copied from HDFS and renamed to `Task_2-1.parquet`. Parquet files must never be combined using `getmerge`, because concatenating them corrupts their footer metadata.

4.4 Results and Validation

The following table summarizes the independently verified results.

Measurement	Result
Input CSV rows	128,975
Distinct promotion identifiers	284
Temporally valid promotions	185
Order IDs with at least 3 valid occurrences	30,503
Normalized non-null state thresholds	40
Cancelled + Standard source rows	6,909
Cancelled + Standard rows with non-null city	6,906
Output city rows	1,434
Qualifying rows	0
Minimum/maximum percentage	0.0 / 0.0

Table 13: Task 2-1 validation results

All 6,909 Cancelled and Standard source rows have a null `promotion-ids` value. Therefore, none can satisfy the requirement of at least three temporally valid promotions. The zero numerator is a property of the data rather than a failed join or empty pipeline.
Grouping before removing null cities would produce 1,435 groups because Spark can form one additional null-city group. Three denominator rows have a null city. Excluding that invalid output key produces 1,434 final city rows.
The generated output was reopened with PyArrow. Its schema is:

```
city: string
percentage: double
```

Independent validation with PyArrow confirmed 1,434 unique non-null city keys and a percentage 0.0. After sorting by the output columns, a row-by-row comparison against the

supplied reference Parquet confirmed exact equality. The schema, comparison result, and a sample of the executed output are included below.

```
PS D:\HK3_3\Big Data\Lab 3> wsl
+----------------------+----------+
|city                  |percentage|
+----------------------+----------+
|GWALIOR               |0.0       |
|HUBBALLI              |0.0       |
|MANDI                 |0.0       |
|MARATHAHALLI,BENGALORE.|0.0       |
|DARBHANGA             |0.0       |
|BASMATH               |0.0       |
|KOLKATA               |0.0       |
|INDORE                |0.0       |
|HAZARIBAGH            |0.0       |
|BARRACKPORE           |0.0       |
|SECUNDERABAD, HYDERABAD|0.0       |
|NEW DELHI             |0.0       |
|THIRUVANANTHAPURAM    |0.0       |
|PENDRA                |0.0       |
|GORAKHPUR             |0.0       |
|VIZIANAGARAM A        |0.0       |
|RAJKOT                |0.0       |
```

Figure 9: Validation of the generated Task 2-1 Parquet output.

4.5 Benchmark

Five independent YARN applications were executed. Each application evaluates the complete lazy pipeline once and overwrites the same HDFS output directory. The result DataFrame is not cached, so every run performs the CSV scan, promotion processing, joins, shuffle aggregations, final `coalesce(1)`, and Parquet write.
The five applications were submitted using the following deployment mode:

```
for TASK21_RUN in 1 2 3 4 5; do
  spark-submit \
    --master yarn \
    --deploy-mode cluster \
    --class Task21 \
    Task21.jar INPUT_PATH OUTPUT_PATH "$TASK21_RUN"
```

```
done
```

The benchmark used **YARN cluster deploy mode** with Spark 4.2.0, Scala 2.13.18, and an HDFS input of 65.7 MB. The Hadoop/YARN installation was still single-node pseudo-distributed under WSL: NameNode, DataNode, SecondaryNameNode, ResourceManager, and NodeManager were separate processes on one host. Thus, "cluster deploy mode" means that the Spark driver ran in the YARN ApplicationMaster container. Hadoop 3.3.6 daemons used Java 11, while the Spark ApplicationMaster and executors used Java 17.
For each application, elapsed time was calculated from the ResourceManager timestamps as

$$\text{elapsedSeconds} = \frac{\text{FinishTime} - \text{StartTime}}{1000}. \tag{3}$$

The `memory-seconds` and `vcore-seconds` values were read directly from YARN's `Aggregate Resource Allocation`. All five applications completed with `Final-State = SUCCEEDED`. The following table summarizes the measurements.

Table 14: Five-application Task 2-1 YARN benchmark

Run	Elapsed (s)	Memory-seconds	vcore-seconds
1	40.310	206,780	99
2	33.006	170,570	81
3	40.449	219,943	105
4	37.062	189,345	90
5	30.568	163,388	78
Mean	36.279	190,005.20	90.60
Sample standard deviation	4.404	23,792.35	11.50

Because every measurement is a new YARN application, elapsed time includes JAR localization, ApplicationMaster and executor startup, pipeline execution, and application shutdown. For example, the fifth application's total elapsed time was 30.568 seconds, while the Scala timer around its final write action reported 14.641 seconds. The difference is YARN and application lifecycle overhead. The aggregate resource counters generally follow elapsed time, which is expected because containers reserve memory and virtual cores for the duration of the application. These figures characterize this single-node WSL cluster and are not hardware-independent performance guarantees.

4.6 Task-Specific Requirements

4.6.1 Structured API Compliance

The solution uses only Spark DataFrame transformations and functions, including `select`, `filter`, `transform`, `explode`, `groupBy`, `agg`, and `join`. It does not call `spark.sql` and contains no SQL query strings.

4.6.2 Extended Execution Plan

The required plan is printed before writing the result:

```
result.explain(true)
```

The Spark 4.2.0 physical plan contains three `BroadcastHashJoin` operators:

1. Inner join between promotion appearances and the 185-row valid promotion relation.
2. Left outer join between the denominator and the per-order qualifying promotion counts.
3. Left outer join between the enriched denominator and the 40 state averages.

Broadcast hash joins are appropriate because the derived relations are small enough to be sent to every executor. The main order relation does not need to be shuffled for these joins. The complete physical-plan section printed by `explain(true)` is included below. Spark abbreviates long scan metadata with ellipses when it renders the plan. Line wrapping is enabled so that every operator remains readable in the final PDF.

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[city#30], functions=[sum(CASE WHEN is_qualifying#176 THEN 1 ELSE 0 END), count(1)], output=[city#30, percentage#195])
   +- Exchange hashpartitioning(city#30, 200), ENSURE_REQUIREMENTS, [plan_id=137]
      +- HashAggregate(keys=[city#30], functions=[partial_sum(CASE WHEN is_qualifying#176 THEN 1 ELSE 0 END), partial_count(1)], output=[city#30, sum#198L, count#199L])
         +- Project [city#30, (((valid_promotion_count#139L >= 3) AND isnotnull(state_merchant_shipped_average#90)) AND (coalesce(amount#32, 0.0) < state_merchant_shipped_average#90)) AS is_qualifying#176]
            +- BroadcastHashJoin [state#31], [state#171], LeftOuter, BuildRight, false, false
               :- Project [city#30, state#31, amount#32, coalesce(valid_promotion_count#85L, 0) AS valid_promotion_count#139L]
               :  +- BroadcastHashJoin [order_id#24], [order_id#127], LeftOuter, BuildRight, false, false
               :     :- Project [trim(Order ID#1, None) AS order_id#24, upper(trim(ship-city#16, None)) AS city#30, upper(trim(ship-state#17, None)) AS state#31, cast(Amount#15 as double) AS amount#32]
               :     :  +- Filter (((isnotnull(trim(Order ID#1, None)) AND (lower(trim(ship-service-level#6, None)) = standard)) AND Contains(lower(trim(Status#3, None)), cancelled)) AND isnotnull(upper(trim(ship-city#16, None))))
               :     :     +- FileScan csv [Order ID#1,Status#3,ship-service-level#6,Amount#15,ship-city#16,ship-state#17] Batched: false, DataFilters: [isnotnull(trim(Order ID#1, None)), (lower(trim(ship-service-level#6, None)) = standard)
, Contain..., Format: CSV, Location: InMemoryFileIndex(1 paths)[hdfs://localhost:9000/user/vandiemmy/lab3/input/Amazon Sale Report.csv], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<Order ID:string,Status:string,ship-service-level:st
ring,Amount:string,ship-city:string,shi...
               :     +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, true]),false), [plan_id=126]
               :        +- Filter (valid_promotion_count#85L >= 3)
               :           +- HashAggregate(keys=[order_id#127], functions=[count(1)], output=[order_id#127, valid_promotion_count#85L])
               :              +- Exchange hashpartitioning(order_id#127, 200), ENSURE_REQUIREMENTS, [plan_id=122]
               :                 +- HashAggregate(keys=[order_id#127], functions=[partial_count(1)], output=[order_id#127, count#201L])
               :                    +- Project [order_id#127]
               :                       +- BroadcastHashJoin [promotion_id#39], [promotion_id#83], Inner, BuildRight, false, false
               :                          :- Filter (length(promotion_id#39) > 0)
               :                          :  +- Generate explode(promotion_ids#137), [order_id#127], false, [promotion_id#39]
               :                          :     +- Project [trim(Order ID#104, None) AS order_id#127, array_distinct(filter(transform(split(coalesce(promotion-ids#123, ), ,, -1), lambdafunction(trim(lambda x_1#36, None), lambda x_1#36, fals
e)), lambdafunction((length(lambda x_2#37) > 0), lambda x_2#37, false))) AS promotion_ids#137]
               :                          :        +- Filter ((isnotnull(trim(Order ID#104, None)) AND isnotnull(cast(gettimestamp(trim(Date#105, None), MM-dd-yy, TimestampType, try_to_date, Some(Etc/UTC), true) as date))) AND (size(array_d
istinct(filter(transform(split(coalesce(promotion-ids#123, ), ,, -1), lambdafunction(trim(lambda x_1#36, None), lambda x_1#36, false)), lambdafunction((length(lambda x_2#37) > 0), lambda x_2#37, false))), false) > 0))
               :                          :           +- FileScan csv [Order ID#104,Date#105,promotion-ids#123] Batched: false, DataFilters: [isnotnull(trim(Order ID#104, None)), isnotnull(cast(gettimestamp(trim(Date#105, None), MM-dd-yy,..
.. Forma
```

Figure 10: Task 2-1 physical execution plan

4.6.3 Exchange and Stage Analysis

The physical plan contains four shuffle Exchanges:

1. Hash partitioning by `promotion_id`.
2. Hash partitioning by `order_id`.
3. Hash partitioning by `state`.
4. Hash partitioning by `city`.

It also contains three `BroadcastExchange` operators, one for each broadcast join. Consequently, seven physical-plan lines contain the word `Exchange`, but only four are shuffle Exchanges.
Using the classroom rule, four shuffle boundaries divide the static pipeline into five stage regions:

$$\text{static stage regions} = 4 + 1 = 5. \tag{4}$$

The fifth benchmark run under Spark 4.2.0 produced 12 distinct runtime stage IDs across eight Spark jobs. This does not contradict the five-region static estimate: AQE, broadcast materialization, and independent broadcast query stages introduce additional runtime jobs and stage IDs. Both values are labelled clearly rather than reporting the static estimate as a measured runtime count.

4.6.4 Single-File Parquet Output

Every benchmark iteration writes the final DataFrame with:

```
result
  .coalesce(1)
  .write
  .mode("overwrite")
  .parquet(outputPath)
```

The resulting submission consists of one valid Parquet file named `Task_2-1.parquet`. The file contains exactly the two required columns and can be read successfully by Spark, Pandas, and PyArrow.

5 Task 2.2

5.1 Problem Statement

5.1.1 Objective and scope

For every SKU within every month, Task 2-2 [**?**] requires two percentile thresholds, **P80 and P90 of the promotion count over the order records**. For each threshold, records whose promotion count is greater than or equal to the threshold are selected, and the **population standard deviation of `Amount`** is computed over them. Every promotion identifier is counted, Amazon-issued ones included. A group with fewer than two qualifying orders after filtering receives a standard deviation of 0.

Two threshold implementations are required: Spark's built-in `percentile_approx` and a self-implemented exact percentile in DataFrame/Dataset operations. The report must compare threshold differences, execution times and the groups whose qualifying sets differ; that evaluation is gathered in Section 5.5.

The solution uses **Scala with the Spark DataFrame API**, packaged as a JAR. No query is written as an SQL string, and nothing depends on the state of a `spark-shell` session. Conditions belonging to other tasks — shipped status, `Qty > 0`, merchant fulfillment, promotion validity — are not added here.

5.1.2 Input data and intended output

The input is `Amazon Sale Report.csv`, stored on HDFS under the shortened name `asr.csv`:

```
hdfs://namenode:9000/lab3/task2-2/input/asr.csv
```

The dataset contains **128,975 records across 24 columns**. Four are used directly: `SKU` identifies the product and `Date` the month, together forming the group key; `promotion-ids` supplies the identifiers to be counted; and `Amount` is the value whose dispersion is measured. `Order ID` is kept in the raw data but is not used as a deduplication key.

The output is an ordinary `Task_2-2.parquet` file, readable by Pandas or Spark in local mode. A `method` column distinguishes the two methods within one file, so each SKU-month group produces two rows. The stored input file is shown in Figure 11.

5.1.3 Formalising the query

Let $G_{s,m}$ be the group for SKU s and month m; each record r has a promotion count $c(r)$ and a possibly missing amount $a(r)$. For $p \in \{0.8, 0.9\}$ the selected set is

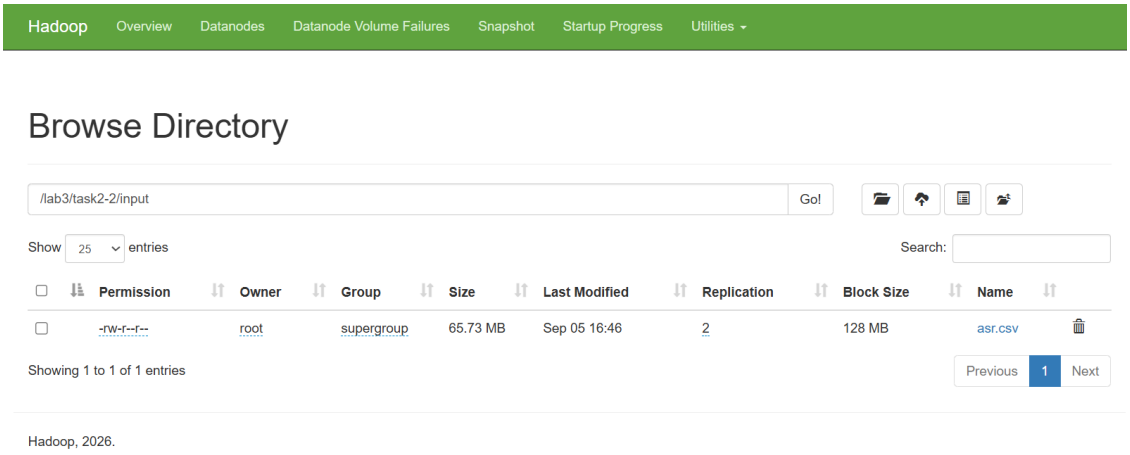

Figure 11: The source CSV file stored on HDFS.

$$S_p(G_{s,m}) = \{r \in G_{s,m} \mid c(r) \geq Q_p(G_{s,m})\}. \tag{5}$$

The promotion count decides which records are selected; `Amount` is only the quantity whose dispersion is then measured. P80/P90 are not computed on `Amount`, no threshold is shared across SKU-months, and exactly 10% or 20% of the rows is not taken — every record whose count equals the threshold is kept.

With k non-null amounts $a_1, \ldots, a_k$ selected and $\bar{a} = \frac{1}{k}\sum_i a_i$:

$$\sigma_p = \begin{cases} 0, & k < 2, \\ \sqrt{\dfrac{1}{k}\sum_{i=1}^{k}(a_i - \bar{a})^2}, & k \geq 2. \end{cases} \tag{6}$$

The denominator is k because this is the population standard deviation. The $k < 2$ branch also covers groups where many records pass the threshold but fewer than two usable amounts remain.

5.1.4 Assumptions and processing conventions

The conventions below state the chosen interpretation and implementation explicitly; not all of them are defined in the assignment.

Topic	Convention adopted by the solution
Unit of observation	One CSV row is one observed record; rows are not merged because `Order ID` repeats.

Topic	Convention adopted by the solution
Group key	`(SKU, year_month)` with `year_month` as `yyyy-MM`, so months from different years are not merged. `Date` is parsed with `MM-dd-yy`.
Counting promotions	The CSV structure is parsed first, then `promotion-ids` is split on commas and the non-empty elements counted. Amazon identifiers are kept and repeats within a cell are not deduplicated.
Missing promotions	Null, an empty string or only empty elements yield a count of 0.
Missing `Amount`	The record still counts towards the threshold and the qualifying count, but a null value never enters the standard deviation.
Numerical handling	Thresholds are not rounded before filtering; an absolute tolerance of 10^{-9} is used only when tallying floating-point differences.

Two counts are stored separately: **the number of records passing the threshold** and **the number of those records that have a monetary value to compute with**. Setting the standard deviation to 0 when there is not enough data is entirely different from replacing a missing `Amount` with 0.

5.2 Problem Decomposition

5.2.1 Decomposition into elementary processing steps

The problem splits into seven steps with a defined input, operation and output. These are logical steps of the application, not physical Spark stages.

Step	Input	Processing	Output
S1 — Read and prepare	CSV on HDFS	Read the header and the quoted fields; derive date, month and amount; count identifiers.	The `prepared` DataFrame.
S2 — Compute per-group thresholds	`prepared`	Group by SKU/month; run the approximate and exact branches separately for P80/P90.	Two threshold tables with the same schema.
S3 — Attach thresholds to records	`prepared` and one threshold table	Join on `(SKU, year_month)`.	Every record carries its group's P80/P90 thresholds.
S4 — Compute statistics	The joined table	Apply the `>=` predicate; count records, count non-null amounts and compute the population standard deviation.	One result table per method.

Step	Input	Processing	Output
S5 — Compare	The two result tables	Compare thresholds, selected record sets and standard deviations.	A detailed table and a summary table of differences.
S6 — Measure performance	The shared prepared input	Warm up, time repeated independent runs of each method and summarise the timings.	The measurements and benchmark statistics.
S7 — Export and read back	The two result tables	Add `method`, union the tables, write Parquet; fetch the file locally and read it back.	One result file plus validation evidence.

S5 and S6 serve the evaluation and do not change the thresholds or the result data produced by S4. S7 runs outside the timed window, so the cost of writing the file never enters the benchmark.

5.2.2 Rationale for this decomposition

Preparation is separated from the percentile algorithm so that both methods share date conversion, identifier counting and null handling; any difference in the output therefore cannot come from reading or cleaning the data differently.

Thresholds are computed per group before rows are selected. P80/P90 belong to the whole group while `promotion_count >= threshold` is evaluated per record, so a threshold table must be built and joined back onto the data.

The statistics step is shared. Counting records, handling missing `Amount` values and computing `stddev_pop` all call the same `AmountStatistics.compute`, which confines the intended difference between the branches to the threshold alone.

P80 and P90 are computed together within each branch: they share a group key and a distribution, and the exact branch reuses one sorted array for both levels.

Each threshold table has the schema `(SKU, year_month, order_count, p80, p90)`, one row per group, with the thresholds typed `double` so both branches share the downstream processing. The result table keeps two counts per level, which separates the records that were selected from the monetary values that actually entered the computation.

5.3 Implementation Strategy

5.3.1 Environment and deployment architecture

The project is developed in VS Code on WSL, where Spark is installed; the Lab 1 Hadoop cluster runs in six Docker containers — `namenode`, two datanodes, `resourcemanager` and

two nodemanagers. In YARN client mode the driver stays on WSL and the executors run in the worker containers. The `local[2]` runs served only development and reading the file back; the benchmark comes from the run whose master is `yarn`.

Item	**Configuration / observation**	**Basis**
Spark / driver Java	3.5.4 / 17.0.20	Execution log.
Hadoop	3.5.0	Cluster configuration.
Project Scala / build tool	2.12.18 / sbt	`build.sbt`.
Master / deploy mode	YARN / client	Submit command and runtime trace.
Executors requested	2 × (1 core, 1,024 MiB)	Submit parameter, not a measured allocation.
Shuffle partitions / AQE	200 / enabled	Printed before the benchmark.
Approximate `accuracy`	10,000	`ApproxPercentile.scala`.

The log records one executor registering on `nodemanager2`. The service layout is shown in Figure 12.

```
qadeptrai123@DESKTOP-LL2B1OG: $ cd ~/hadoop-cluster-config && sudo docker compose ps
[sudo] password for qadeptrai123:
NAME             IMAGE                    COMMAND                  SERVICE          CREATED        STATUS                     PORTS
datanode1        hadoop-apache:3.5.0   "bash /opt/bootstrap…"   datanode1        29 hours ago   Up About an hour           22/tcp, 8042/tcp, 8088/tcp, 9000/tcp, 98
66-9867/tcp, 9870/tcp, 19888/tcp, 0.0.0.0:9864->9864/tcp, [::]:9864->9864/tcp
datanode2        hadoop-apache:3.5.0   "bash /opt/bootstrap…"   datanode2        29 hours ago   Up About an hour           22/tcp, 8042/tcp, 8088/tcp, 9000/tcp, 98
66-9867/tcp, 9870/tcp, 19888/tcp, 0.0.0.0:9865->9864/tcp, [::]:9865->9864/tcp
namenode         hadoop-apache:3.5.0   "bash /opt/bootstrap…"   namenode         29 hours ago   Up About an hour (healthy) 22/tcp, 8042/tcp, 8088/tcp, 9864/tcp, 0.
0.0.0:9000->9000/tcp, [::]:9000->9000/tcp, 9866-9867/tcp, 19888/tcp, 0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp
nodemanager1     hadoop-apache:3.5.0   "bash /opt/bootstrap…"   nodemanager1     29 hours ago   Up About an hour           22/tcp, 8088/tcp, 9000/tcp, 9864/tcp, 98
66-9867/tcp, 9870/tcp, 19888/tcp, 0.0.0.0:8042->8042/tcp, [::]:8042->8042/tcp
nodemanager2     hadoop-apache:3.5.0   "bash /opt/bootstrap…"   nodemanager2     29 hours ago   Up About an hour           22/tcp, 8088/tcp, 9000/tcp, 9864/tcp, 98
66-9867/tcp, 9870/tcp, 19888/tcp, 0.0.0.0:8043->8042/tcp, [::]:8043->8042/tcp
resourcemanager  hadoop-apache:3.5.0   "bash /opt/bootstrap…"   resourcemanager  29 hours ago   Up About an hour (healthy) 22/tcp, 8042/tcp, 9000/tcp, 9864/tcp, 98
66-9867/tcp, 9870/tcp, 19888/tcp, 0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp
qadeptrai123@DESKTOP-LL2B1OG:~/hadoop-cluster-config$
```

Figure 12: Hadoop services reused from the Lab 1 environment.

5.3.2 Project and source organisation

`build.sbt` pins Spark 3.5.4 and Scala 2.12.18 with the Spark dependency declared `provided`; the application is packaged by `sbt package` and run by `spark-submit`. The pipeline is split across nine source files, one responsibility each:

File	**Implementation responsibility**
`Main.scala`	Accept the input/output paths, create the SparkSession, coordinate the steps and call `spark.stop()` in `finally`.
`DataPreparation.scala`	Build the prepared columns using DataFrame expressions.
`ApproxPercentile.scala`	Compute P80/P90 with the built-in approximate function.

File	Implementation responsibility
`ExactPercentile.scala`	Collect the promotion counts per group, sort them and interpolate.
`AmountStatistics.scala`	Join the threshold table, count, and compute the population standard deviation.
`Comparison.scala`	Build the difference tables per group and per percentile level.
`Benchmark.scala`	Warm up, time repeated runs, alternate the method order and summarise the timings.
`ResultWriter.scala`	Union the two methods and write a single Parquet part-file.
`VerifyOutput.scala`	Read the local file and check the row count and key uniqueness.

Figure 13 illustrates the standard sbt directory layout with the nine source files described above. Resource and test directories are optional; `target/` contains generated build output.

```
Task_2-2/
├── build.sbt
├── project/
│   └── build.properties
├── src/
│   └── main/
│       └── scala/
│           ├── Main.scala
│           ├── DataPreparation.scala
│           ├── ApproxPercentile.scala
│           ├── ExactPercentile.scala
│           ├── AmountStatistics.scala
│           ├── Comparison.scala
│           ├── Benchmark.scala
│           ├── ResultWriter.scala
│           └── VerifyOutput.scala
└── target/ ..... generated by sbt
    └── scala-2.12/
        └── task-2-2_2.12-0.1.0.jar
```

Figure 13: Standard sbt layout for Task 2.2, showing the Scala sources and generated JAR.

5.3.3 Data preparation

The CSV reader uses `header = true`, `inferSchema = false` and `mode = FAILFAST`. Once parsed, `DataPreparation.prepare` derives four columns:

Derived column	How it is built
`order_date`	`to_date(col("Date"), "MM-dd-yy")`.
`year_month`	`date_format(col("order_date"), "yyyy-MM")`.
`amount_value`	`Amount` cast to `double`.
`promotion_count`	Count of the non-empty elements in `promotion-ids`.

The counting logic:

```
// Count non-empty identifier occurrences in the parsed CSV field.
val promotionIds = split(coalesce(col("promotion-ids"), lit("")), ",")
val nonEmptyIds = filter(promotionIds, id => length(trim(id)) > 0)
val promotionCount = size(nonEmptyIds)
```

`filter` removes empty elements from the identifier array, not order rows, and `coalesce` replaces only a null identifier field — a missing `Amount` stays null. Splitting after the CSV has been parsed avoids confusing a comma inside a quoted cell with a column separator.

5.3.4 Implementing the approximate threshold

`ApproxPercentile.compute` groups by `SKU` and `year_month`, computes `order_count` with `count(lit(1))` and computes both thresholds at once:

```
// Compute both thresholds from the same promotion-count distribution.
percentile_approx(
  col("promotion_count"),
  array(lit(0.8), lit(0.9)),
  lit(10000)
)
```

The returned array is read in the order `[P80, P90]`. The values are cast to `double` and named `p80` and `p90`. The same `accuracy = 10000` is kept for every measured run; it is not tuned per percentile level or per run.

5.3.5 Implementing the exact threshold

`ExactPercentile.compute` uses `collect_list` and `sort_array` to produce the ascending sequence of promotion counts within each group:

$$x_0 \leq x_1 \leq \cdots \leq x_{n-1}. \tag{7}$$

For each $p \in \{0.8, 0.9\}$, the interpolation position is:

$$h = (n-1)p, \qquad i = \lfloor h \rfloor, \qquad j = \lceil h \rceil, \tag{8}$$

$$Q_p = x_i + (h - i)(x_j - x_i). \tag{9}$$

The interpolation is built from `Column` expressions; no built-in exact-percentile function and no UDF is used. When indexing with `element_at`, the index is increased by one because the formula uses zero-based positions. If $n = 1$, both indices point to the single element.
`collect_list` is used rather than `collect_set` because repeated values are still distinct observations. Deduplicating them would change their weight in the promotion distribution. The array is collected per group inside Spark; the source records are not brought back to the driver.
For the chosen algorithm alone, sorting a group of n elements costs $O(n \log n)$ and stores $O(n)$ elements. The sorting term across all groups is $O(\sum_G |G| \log |G|)$. This is an algorithmic analysis of the solution, not a measurement of actual memory use, stage count or shuffle volume.

5.3.6 Applying the threshold and computing shared statistics

`AmountStatistics.compute(prepared, thresholds)` joins the two tables on the key `(SKU, year_month)` with an `inner join`. The threshold table is derived from the very same prepared input; for groups with a valid key, each group has exactly one matching threshold row. The experimental result on group coverage is cross-checked in Section 5.4.
After the join, the program regroups by the key and the thresholds and aggregates conditionally. For P80, the central expressions are:

```
val qualifiesP80 = col("promotion_count") >= col("p80")

// Count qualifying rows, including rows with missing amounts.
val selectedCount = count(when(qualifiesP80, lit(1)))

// Count only qualifying rows with a non-null amount.
val amountCount = count(when(qualifiesP80, col("amount_value")))

// Nonqualifying rows contribute null, not zero.
val amountStddev = coalesce(
  stddev_pop(when(qualifiesP80, col("amount_value"))),
  lit(0.0)
)
```

P90 follows the same pattern. A record that does not pass the threshold contributes null to the amount expression, not the value 0. `stddev_pop` computes the population standard deviation; `coalesce` turns a null result into 0 when no usable amount remains. This handling is identical across both branches.

5.3.7 Implementing the evaluation and the export

`Comparison.compute` takes the two statistics tables and one level, builds the detailed table and summarises the groups differing in threshold, in selected set and in standard deviation; the results appear in Sections 5.5.1–5.5.2.
`ResultWriter` adds `method = "approx"` or `"exact"`, unions the tables with `unionByName` and writes Parquet with `coalesce(1)` and `maxRecordsPerFile = 0`; `overwrite` applies only to the designated output directory. One part-file is fetched to WSL as `Task_2-2.parquet`; `getmerge` is never used on Parquet.
`VerifyOutput` is a separate program: it requires a regular file, reads it with Spark local and checks the row count against the number of distinct `(SKU, year_month, method)` keys.

5.3.8 Build, run and reproduction

Run from the directory containing `build.sbt`:

```
sbt package
```

The JAR produced by the project configuration:

```
target/scala-2.12/task-2-2_2.12-0.1.0.jar
```

The environment needs working HDFS/YARN services, adequate HDFS permissions and the ability to resolve container hostnames from WSL. Configure the Hadoop directory and the driver address from the current Docker bridge:

```
export HADOOP_CONF_DIR="$HOME/hadoop-cluster-config/conf"
DRIVER_IP=$(sudo docker network inspect hadoop-cluster-config_hadoopnet -f '{{(index .
    IPAM.Config 0).Gateway}}')
mkdir -p evidence
```

The submit command reproducing the configuration used during implementation:

```
spark-submit \
  --class Main \
```

```
  --master yarn \
  --deploy-mode client \
  --num-executors 2 \
  --executor-cores 1 \
  --executor-memory 1g \
  --driver-memory 1g \
  --conf spark.dynamicAllocation.enabled=false \
  --conf spark.driver.host="$DRIVER_IP" \
  --conf spark.driver.bindAddress=0.0.0.0 \
  target/scala-2.12/task-2-2_2.12-0.1.0.jar \
  hdfs://namenode:9000/lab3/task2-2/input/asr.csv \
  hdfs://namenode:9000/lab3/task2-2/output/result-yarn \
  2>&1 | tee evidence/run-yarn-rerun.log
```

Redirecting to a separate log file keeps the recorded run intact. `ResultWriter` opens the output directory in `overwrite` mode, so an earlier result must be copied elsewhere first if it is still needed.

5.4 Results and Validation

5.4.1 Evidence that the application ran on YARN

The application was submitted to the ResourceManager in YARN client mode and completed with state `FINISHED` and final status `SUCCEEDED`, as shown in Figure 14. Executors ran on the cluster nodemanagers; the program prints `Execution master: yarn`, so the pipeline did not fall back to a local Spark session reading an HDFS file.

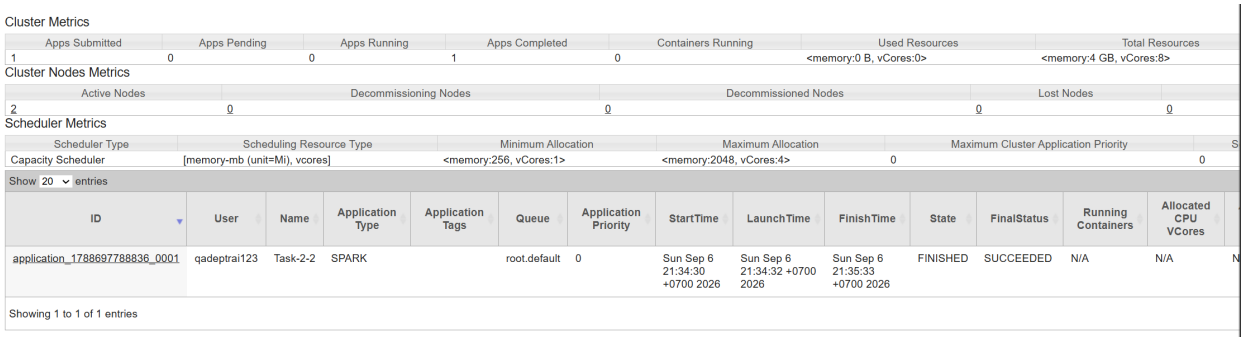

Figure 14: The application on the ResourceManager after completion.

Application ID: `application_1788697788836_0001`
State / final status: `FINISHED / SUCCEEDED`

5.4.2 Group coverage and result consistency

The YARN run records:

Metric	Result
Number of `(SKU, year_month)` groups	16,486
Records in the largest group	426
Groups with more than 1,000 records	0
Groups compared at P80	16,486
Groups compared at P90	16,486

The number of groups in both comparison tables equals the number of groups counted on the input, which supports the group-coverage check on the current data.
The detailed tables on thresholds and changed records are placed in Section 5.5, so that **output validation** is kept separate from the **dedicated comparison between the two methods**. The image in Figure 17 in Section 5.5.1 also contains these group statistics.

5.4.3 Schema and output file

The combined result has **12 columns**, a row identified by `(SKU, year_month, method)`:

Column	Type	Meaning
`SKU`	string	The original SKU key.
`year_month`	string	Month in `yyyy-MM` form.
`order_count`	long	All source records in the group.
`p80`	double	P80 threshold of the promotion count.
`p90`	double	P90 threshold of the promotion count.
`p80_order_count`	long	Records passing P80, including those with a missing amount.
`p80_amount_count`	long	Records passing P80 with a non-null amount.
`p80_stddev`	double	Population standard deviation of the amounts after the P80 filter.
`p90_order_count`	long	Records passing P90, including those with a missing amount.
`p90_amount_count`	long	Records passing P90 with a non-null amount.
`p90_stddev`	double	Population standard deviation of the amounts after the P90 filter.
`method`	string	`approx` or `exact`.

The final log message reports the write to HDFS:

```
Results written to: hdfs://namenode:9000/lab3/task2-2/output/result-yarn
```

The part-file is then fetched to the local filesystem:

```
sudo docker exec namenode /opt/hadoop/bin/hdfs dfs -get -f '/lab3/task2-2/output/result
    -yarn/part-*.parquet' /tmp/Task_2-2.parquet
sudo docker cp namenode:/tmp/Task_2-2.parquet ./Task_2-2.parquet
```

The deliverable is an actual Parquet file, not a Spark output directory renamed with a `.parquet` extension. Figure 15 shows the result directory and the local file.

```
qadeptrai123@DESKTOP-LL2B1OG:~/spark-lab3/task-2-2$ sudo docker exec namenode /opt/hadoop/bin/hdfs dfs -ls /lab3
/task2-2/output/result-yarn/
ls ~/spark-lab3/task-2-2/Task_2-2.parquet
Found 2 items
-rw-r--r--   2 qadeptrai123 supergroup          0 2026-09-06 14:35 /lab3/task2-2/output/result-yarn/_SUCCESS
-rw-r--r--   2 qadeptrai123 supergroup     341515 2026-09-06 14:35 /lab3/task2-2/output/result-yarn/part-00000-0
2508588-76ca-4934-8e35-f13649605dd5-c000.snappy.parquet
/home/qadeptrai123/spark-lab3/task-2-2/Task_2-2.parquet
qadeptrai123@DESKTOP-LL2B1OG:~/spark-lab3/task-2-2$
```

Figure 15: The Spark part-file and the standalone Parquet deliverable.

5.4.4 Validating the file with Spark local

The command that reads back the very file prepared for submission:

```
spark-submit \
  --class VerifyOutput \
  --master "local[2]" \
  target/scala-2.12/task-2-2_2.12-0.1.0.jar \
  ./Task_2-2.parquet
```

The terminal output, stored separately from the YARN log:

```
method   count
exact    16486
approx   16486

Verified rows: 32972
PARQUET_READBACK_OK
```

The row total matches $2 \times 16{,}486 = 32{,}972$. As `VerifyOutput` is implemented, the program prints the success message only after checking that the path is a regular file, that the file can be read, that it has exactly 32,972 rows, and that the number of distinct keys equals the

number of rows. This result therefore supports readability, row count and key uniqueness for the chosen schema. This output is shown in Figure 16.

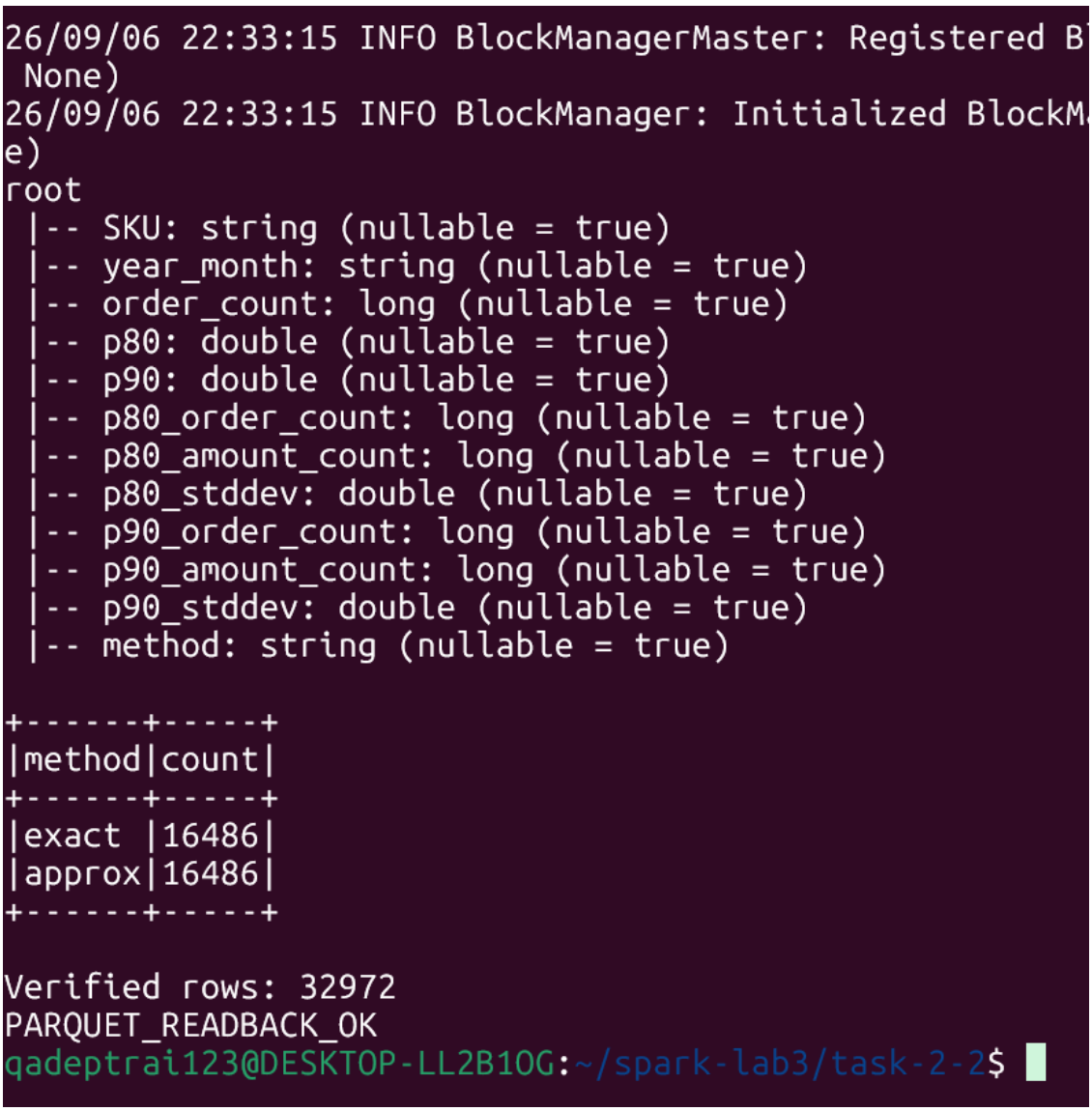

Figure 16: Structural and readability validation of the Parquet file with Spark local.

5.5 Percentile Method Comparison and Partition Strategy — Task 2-2 Specific Requirements

This section addresses the requirements specific to Task 2-2: **the difference between the approximate and exact thresholds, execution time, and the groups whose qualifying record sets differ**. The assignment [?] additionally requires a partitioning discussion if any group exceeds 1,000 orders. The rule of at least five measurements, reporting

both the mean and the standard deviation, belongs to the general lab requirements.

5.5.1 Threshold differences and interpreting accuracy

Metrics

For each group G and level p the program computes the absolute differences $\Delta Q_p(G) = \left|Q_p^{\text{approx}}(G) - Q_p^{\text{exact}}(G)\right|$ and $\Delta\sigma_p(G)$ between the two standard deviations. A group counts as differing when the difference exceeds 10^{-9}; this tolerance is used only when tallying floating-point differences and never changes the record filter.

Results over all groups

Metric	P80	P90
Total groups	16,486	16,486
Groups with differing thresholds	6,053	7,168
Percentage with differing thresholds	36.72%	43.48%
Groups with differing selected record sets	1,002	305
Percentage with differing record sets	6.08%	1.85%
Groups with differing final standard deviations	403	125
Percentage with differing final standard deviations	2.44%	0.76%
Mean absolute threshold difference	0.9355817057	1.0303590926

Percentages are the differing-group counts divided by 16,486. Thresholds differ far more often than final results: 36.72% against 2.44% at P80. Thresholds of 0.8 and 1.0 retain the same integer promotion counts, so the threshold differs while the selected set does not. Figure 17 shows the program output.

```
qadeptrai123@DESKTOP-LL2B1OG:~/spark-lab3/task-2-2$ sed -n '114,127p' ~/spark-lab3/task-2-2/evidence/run-yarn-ne
w.log
SKU-month group statistics:
+------------+-------------+----------------+
|total_groups|largest_group|groups_over_1000|
+------------+-------------+----------------+
|16486       |426          |0               |
+------------+-------------+----------------+

Comparison summary:
+----------+------------+----------------------+-----------------+--------------------+--------------------+
|percentile|total_groups|threshold_changed_groups|set_changed_groups|stddev_changed_groups|mean_threshold_diff|
+----------+------------+----------------------+-----------------+--------------------+--------------------+
|P80       |16486       |6053                  |1002             |403                 |0.935581705689688   |
|P90       |16486       |7168                  |305              |125                 |1.030359092563401   |
+----------+------------+----------------------+-----------------+--------------------+--------------------+
qadeptrai123@DESKTOP-LL2B1OG:~/spark-lab3/task-2-2$
```

Figure 17: Group coverage and the differences between approximate and exact.

Approximation error versus definitional difference

The gap is not attributed to numerical error in Spark alone, because the two methods apply different percentile conventions. On the sequence `[0, 0, 1, 1, 2, 2, 2, 3, 5, 9]` linear interpolation gives P90 = 5.4 whereas nearest-rank gives 5. The experiment therefore compares two implementations under their stated definitions rather than isolating approximation error against a baseline using the same rank convention.

Exact thresholds can be fractional even though promotion counts are integers; values such as `1.400000000000034` are kept unrounded during filtering and rounded only for presentation.

5.5.2 Analysis of groups with differing qualifying record sets

Let A and E be the record sets selected by the two thresholds on the same group. Both apply `promotion_count >= threshold`, so the set chosen by the higher threshold is a subset of the other, and $|A \triangle E| = \big||A| - |E|\big|$. `Comparison.scala` uses this to derive `changed_order_count` from the two counts; it is a count of records changing selection state, not of distinct `Order ID` values.

A representative P90 case is `JNE3567-KR-L` in April 2022, containing 120 records:

Quantity	Approximate	Exact
P90 threshold	1.0	Approximately 1.4
Records selected	57	12
Population standard deviation	54.378606495457	112.379905034466

Raising the threshold from 1 to about 1.4 excludes the 45 records holding exactly one promotion identifier. The selected set shrinks yet the standard deviation rises by about 58.001299 — a small threshold change matters greatly when many records sit on the boundary. The result also matches the rounded example published in [1, PDF p. 26].

At P80, `J0119-TP-XXL` in June 2022 raises its threshold from 0 to about 0.2. The number of selected records falls from 35 to 7 while the population standard deviation rises from about 72.060336 to 99.251073; with integer counts the new threshold excludes every record holding no identifier. A differing record set does not always change the standard deviation: the remaining amounts may be identical, or too few. Figure 18 shows the detailed comparison table.

5.5.3 Execution-time comparison

Protocol

All timings below come from the run recorded in `evidence/run-yarn-new.log`, the same

```
qadeptrai123@DESKTOP-LL2B1OG:~/spark-lab3/task-2-2$ sed -n '129,144p' ~/spark-lab3/task-2-2/evidence/run-yarn-new.log
Groups with different selected orders:
+----------------+----------+----------+-----------+----------------+------------------+--------------+--------------+-----------------+------------------+--------
----------+-----------------+-----------------+
|SKU             |year_month|percentile|order_count|approx_threshold|exact_threshold   |approx_selected|exact_selected|approx_stddev    |exact_stddev      |threshol
d_diff    |changed_order_count|stddev_diff      |
+----------------+----------+----------+-----------+----------------+------------------+--------------+--------------+-----------------+------------------+--------
----------+-----------------+-----------------+
|JNE3567-KR-L    |2022-04   |P90       |120        |1.0             |1.400000000000034 |57            |12            |54.378606495457  |112.37990503446582|0.400000
0000000341 |45               |58.00129853900882 |
|SET397-KR-NP  -M |2022-05   |P80       |35         |1.0             |2.200000000000017 |62            |17            |135.610943672347 |235.058823529411 74|1.200000
000000017  |45               |99.44787985706475 |
|J0119-TP-XXL    |2022-06   |P80       |35         |0.0             |0.2000000000000284|35            |7             |72.06033611857843 |99.25107309346373 |0.200000
00000000284|28               |27.190736974885297|
|JNE3465-KR-XXL  |2022-04   |P90       |60         |1.0             |2.700000000000024 |34            |6             |82.64257890307432 |0.0               |1.700000
0000000242 |28               |82.64257890307432 |
|J0344-TP-L      |2022-04   |P80       |30         |0.0             |0.2000000000000284|30            |6             |87.28273576843375 |6.70820393249937  |0.200000
00000000284|24               |80.57453183593438 |
|PJNE3373-KR-N-6XL|2022-05   |P80       |25         |0.0             |0.2000000000000284|25            |5             |201.5327206683818 |16.000000000000004|0.200000
00000000284|20               |185.5327206683818 |
|JNE3800-KR-L    |2022-04   |P80       |25         |1.0             |1.8000000000000114|24            |5             |153.68473660025217|308.4             |0.800000
0000000114 |19               |154.71526339974 78|
|JNE3721-KR-XL   |2022-05   |P80       |60         |1.0             |4.2000000000000455|30            |12            |6.448427887649993 |4.4999999999999964|3.200000
0000000455 |18               |1.9484278876499967|
|JNE2014-KR-178-L|2022-04   |P90       |50         |1.0             |2.3000000000000185|22            |5             |8.870975982158516 |9.2               |1.300000
0000000185 |17               |0.3290240178414834|
|J0198-TP-M      |2022-05   |P80       |20         |0.0             |0.2000000000000107|20            |4             |135.81843578837152|0.0               |0.200000
00000000107|16               |135.81843578837152|
+----------------+----------+----------+-----------+----------------+------------------+--------------+--------------+-----------------+------------------+--------
----------+-----------------+-----------------+
only showing top 10 rows
qadeptrai123@DESKTOP-LL2B1OG:~/spark-lab3/task-2-2$
```

Figure 18: Detailed comparison of groups with differing qualifying record sets.

application shown in Figure 14. `Benchmark.scala` runs two warm-up rounds and five measured runs per method, alternating the order between rounds. The shared input is projected onto the four required columns, cached and materialised before timing; each measured run rebuilds that method's query and collects the 16,486-group aggregated table to the driver. `System.nanoTime()` brackets that call, so the measured window covers query construction, both thresholds, the join, the `Amount` statistics and the transfer to the driver — but not Spark/YARN startup, reading and preparing the CSV, materialising the cache, or anything after the benchmark. This is a warm cached-input benchmark, not end-to-end latency.

Measurements

All times in seconds; the decimal point is kept so the values match the log directly.

Run	Execution order	Approximate	Exact
1	Approximate then exact	0.57014361	0.585678923
2	Exact then approximate	0.565108243	0.478880251
3	Approximate then exact	0.556404073	0.536946948
4	Exact then approximate	0.545293973	0.531537205
5	Approximate then exact	0.487808553	0.516041322
Mean		**0.5449516904**	**0.5298169298**
Sample std. dev.		**0.0333074008**	**0.0385962155**

The summary values match recomputation from the five measurements of each method; Figure 19 shows the printed benchmark. For the five times $t_1, \ldots, t_5$ the program uses the arithmetic mean and the **sample** standard deviation $s_t = \sqrt{\sum_i (t_i - \bar{t})^2/4}$. `stddev_samp` covers run-to-run timing variation while `stddev_pop` remains in use for the order amounts; the assignment requires a mean and a standard deviation but does not fix the formula.

```
qadeptrai123@DESKTOP-LL2B1OG:~/spark-lab3/task-2-2$ sed -n '146,183p' ~/spark-lab3/task-2-2/evidence/run-yarn-new.log
Benchmark master: yarn
Spark version: 3.5.4
Shuffle partitions: 200
AQE enabled: true
Running warm-up rounds...
approx run 1: 0.57014361 seconds
exact run 1: 0.585678923 seconds
exact run 2: 0.478880251 seconds
approx run 2: 0.565108243 seconds
approx run 3: 0.556404073 seconds
exact run 3: 0.536946948 seconds
exact run 4: 0.531537205 seconds
approx run 4: 0.545293973 seconds
approx run 5: 0.487808553 seconds
exact run 5: 0.516041322 seconds
Individual benchmark measurements:
+------+---+---------------+-----------+
|method|run|execution_order|seconds    |
+------+---+---------------+-----------+
|approx|1  |1              |0.57014361 |
|exact |1  |2              |0.585678923|
|exact |2  |1              |0.478880251|
|approx|2  |2              |0.565108243|
|approx|3  |1              |0.556404073|
|exact |3  |2              |0.536946948|
|exact |4  |1              |0.531537205|
|approx|4  |2              |0.545293973|
|approx|5  |1              |0.487808553|
|exact |5  |2              |0.516041322|
+------+---+---------------+-----------+

Benchmark summary:
+------+-------------+------------+-------------------+
|method|measured_runs|mean_seconds|stddev_seconds     |
+------+-------------+------------+-------------------+
|approx|5            |0.5449516904|0.03330740083810112|
|exact |5            |0.5298169298|0.03859621551398112|
+------+-------------+------------+-------------------+
qadeptrai123@DESKTOP-LL2B1OG:~/spark-lab3/task-2-2$
```

Figure 19: Five measured runs per method and execution-time statistics on YARN.

Observations

The difference in means is **0.015134761 s**, i.e. 15.13 ms, or

$$\left(\frac{0.5298169298}{0.5449516904} - 1\right) \times 100 \approx -2.78\%. \tag{10}$$

On this run the exact method is therefore about **2.78%** faster on average, a difference smaller than the sample standard deviation of either method (0.0333 and 0.0386 s). This is a descriptive result from five measurements per method; it does not establish statistical significance or a general performance advantage, and it is limited to this dataset, this configuration and the warm cached-input protocol.

5.5.4 Group size and partition strategy

The assignment requires a discussion of manual repartitioning, of the chosen partition strategy and of the 128 MB partition size **only if some SKU-month group exceeds 1,000 orders**. In this run the largest group holds 426 records and no group exceeds the threshold, so the condition does not arise.

Accordingly, no manual `repartition` is added before the percentile computations, and the run keeps `spark.sql.shuffle.partitions = 200` with `spark.sql.adaptive.enabled =`

`true`. The exact branch collects only the promotion counts within each group, so the largest array holds 426 elements. `coalesce(1)` is applied only when exporting the aggregated table, outside the benchmark.
The log measures **records per group**, not the serialised size of a group, and contains no final physical plan or per-stage task counts. The value 200 is therefore reported as a configuration setting rather than as a measured task count, and no byte ratio against 128 MB is claimed.

5.5.5 Conclusion

Both methods are implemented in a Scala project that builds and runs, sharing the record-selection and statistics logic. Across 16,486 SKU-month groups the thresholds differ far more often than the final statistic, and the two methods differ in mean execution time by less than one sample standard deviation. These outcomes hold for the interpolation definition, the data and the configuration described above.

References

[1] "Lab 3 — MapReduce & Spark," Course reference slides, Introduction to Big Data Analysis, University of Science, VNUHCM, Aug. 2026, `Lab3_Slide_ref.pdf`, 26 slides. [Online]. Available: https://drive.google.com/file/d/1qow037JXpgGJD4XVZz6rqpThiqYFi3fy/view?usp=sharing

A Appendix

- This LaTeX template is freely provided by Quan, Tran Hoang at `https://github.com/khongsomeo/hcmus-unofficial-report-template`, with modifications provided by the project's authors under the GNU GPL v3.0 license.